



Instruções: Esta prova está dividida em duas partes: questões de cunho teórico e questões de aplicação prática dos conceitos em compiladores. Responda as 10 (dez) questões da prova exclusivamente na folha de respostas pautada que acompanha esta folha de questões; nenhuma resposta indicada nesta folha de questões será considerada; identifique cada questão com o seu respectivo número na folha de respostas; respostas à lápis não darão direito à revisão da correção; assine também a folha de respostas. Qualquer questão tem valor de 1 (um) ponto.

Parte teórica

1. Desenvolva e apresente uma gramática livre de contexto (GLC), G , para a seguinte linguagem: $\{w \in \{a, b\}^* \mid w = a^n b^n, \text{ para } n \geq 0\}$. Indique na sua resposta quais são os símbolos não-terminais e qual deles é o símbolo de partida, assim como as respectivas regras de produção de G .
2. Para a gramática G , apresentada como resposta da questão anterior, demonstre o processo de derivação (por construção de árvore sintática ou por formas sentenciais), a partir do símbolo de partida, para 2 (dois) exemplos de cadeias válidas da linguagem geradas pela gramática.
3. Ainda considerando a gramática G das questões anteriores, apresente uma outra gramática G' , que seja equivalente a G . Na sua resposta, indique as mesmas informações para G' que foram solicitadas para a apresentação de G .
4. Demonstre que a gramática de operadores lógicos a seguir é ambígua. $GL = (\{v, f, e, ou\}, \{X\}, P, X)$, com $\{v, f, e, ou\}$ o conjunto dos símbolos terminais, no qual "ou" e "e" são operadores e "v" e "f" são operandos da linguagem, $\{X\}$ é o conjunto de símbolos não-terminais, X é o símbolo de partida e P é formado pelas seguintes regras de produção: $X \rightarrow X \text{ e } X \mid X \text{ ou } X \mid v \mid f$.

- Para a gramática GL anterior, encontre e apresente uma GL' não ambígua, equivalente a GL , que, adicionalmente, implementa a precedência do operador “e” maior que a precedência do operador “ou” nas árvores sintáticas produzidas para cadeias válidas da linguagem.
- Sabendo-se que gramática GL anterior possui como característica a recursividade à esquerda na montagem de suas árvores sintáticas e que, genericamente, se a gramática possui regras de produção do tipo $A \rightarrow A\alpha \mid \beta$, estas podem ser transformadas em $A \rightarrow \beta R$ e $R \rightarrow \alpha R \mid \epsilon$, encontre GL'' equivalente, a partir de GL , aplicando esses passos, de modo que GL'' não seja recursiva à esquerda. Não se preocupe, nessa transformação, se GL'' permanecer ambígua tal qual GL .

Parte práctica

7. Seja GD a gramática que gera **cadeias de caracteres** que representam números naturais no sistema decimal, definida como $GD = (\{0', 1', 2', 3', 4', 5', 6', 7', 8', 9'\}, \{S, N\}, P, S)$, com $\{0', 1', 2', 3', 4', 5', 6', 7', 8', 9'\}$, o conjunto dos símbolos terminais, $\{S, N\}$ o conjunto de símbolos não-terminais, S o símbolo de partida e P as regras de produção no formato: $S \rightarrow SN \mid N$ e $N \rightarrow 0' \mid 1' \mid \dots \mid 9'$. Apresente uma possível definição dirigida pela sintaxe (DDS), ou seja, as regras semânticas para cada regra de produção de GD, que converta qualquer cadeia de caracteres gerada por GD no seu respectivo número decimal.

8. A gramática de uma determinada linguagem de programação possui o comando condicional **if** apresentado a partir dos seguintes fragmentos do seu conjunto de regras de produção: $\langle \text{cmd} \rangle \rightarrow \text{if } \langle \text{expr_cond} \rangle \text{ then } \langle \text{bloco_cmds} \rangle$, $\langle \text{cmd} \rangle \rightarrow \text{if } \langle \text{expr_cond} \rangle \text{ then } \langle \text{bloco_cmds}_1 \rangle \text{ else } \langle \text{bloco_cmds}_2 \rangle$. Ou seja, só é possível saber se um comando **if** terá um **else** associado ou não após processar toda a sua parte inicial. Usando a técnica da fatoração de regras de produção, apresente um novo conjunto de regras de produção para o comando **if** dessa linguagem de modo que resolva esse problema.

9. Na linguagem Do::Block, o comando **while** está assim definido na especificação de sua gramática: $\text{cmd} ::= \text{while } (\text{expr}) \{ \text{cmd} \} \text{ endwhile}$. Apresente o código da função em linguagem C que pode ser usada no processo de análise sintática descendente recursiva para esse comando. Utilize as chamadas do analisador léxico necessárias tal qual você usou no seu próprio projeto do compilador de Do::Block.

10. Traduza o trecho de código Do::Block a seguir para o respectivo código da máquina de pilha, como se gerado por esquemas de tradução fosse, em um processo de compilação por tradução dirigida pela sintaxe que aplica o modelo descendente recursivo. Como referência, caso necessite, as regras de produção do comando **if** e do comando de atribuição são as seguintes:

$\text{cmd} ::= \text{if } (\text{expr}) \{ \text{cmd} \} \{ \text{elseif } (\text{expr}) \{ \text{cmd} \} \} [\text{else } \{ \text{cmd} \}] \text{ endif}$

$atrib ::= \mathbf{id} \{ [expr] \} = expr$

```
if (a > b)
    c = a + b
else
    c = a - b
endif
```

Sucesso!